AI Powered Customer Support System

System Analysis & Design Report

Prepared By:

Mahmoud Mohamed El-Saeed

Course Name:

Alaa Essam ML & AI Discovery Camp

Instructor:

Eng. Mostafa Mohamed

Eng. Ayman ElGamal

Department:

Computer Engineering

Submission Date:

27-2-2026

AI Powered Customer Support System

◆ System Analysis

1.Business Problem

- The company is facing operational challenges including:
  - ➢ High Ticket Volume
  - ➢ Manual Processing Bottlenecks
  - ➢ Lack of Intelligent Prioritization
  - ➢ Slow Response Time
  - ➢ Repetitive Customer Queries
  - ➢ No Data-Driven Insights
- **Engineering Goal:**

  Design an AI Augmented Decision Support System that:
  - ➢ Reduces manual workload
  - ➢ Improves response efficiency
  - ➢ Enhances customer satisfaction
  - ➢ Enables scalable operations

2.Stakeholder Analysis

Stakeholder	Goal	System Value
Customer	Fast Response	Automated Analysis
Support Agent	Reduce Workload	Ai Suggestions
Manager	Monitor Performance	Analytics Dashboard
Company	Reduce Cost	Automation & Insights

➢ **The system is a support tool, not a replacement for humans (Human in the Loop design).**

3.Functional Requirements

- Ticket Ingestion
- NLP Text Processing

- AI Prediction Pipeline
- Knowledge Retrieval
- Ticket Ranking Engine
- Agent Assistance Panel
- Performance Analytics

4.Non-Functional Requirements

- Scalability → Microservice Ready
- Availability → 24/7 uptime
- Latency → Less than 2 sec prediction
- Security → Data Encryption
- Maintainability → Modular AI models
- Observability → Logging & Monitoring

➢ **Why are Non-Functional Requirements important?**
→ Because they define the real world quality of the system, not just its functionality.

5.System Inputs

- The Ai Ticket System receives the following inputs:
  - Ticket Title (Text)
  - Ticket Description (Text)
  - Timestamp (auto-generated)

➢ **The system combines the title and description into a unified text field for NLP processing.**

6.Systems Outpts

- For each submitted ticket, the system returns:
  - Predicted Category
    (Account / Technical / Billing / Delivery)

- Predicted Priority
  (Low / Medium / High)
- Predicted Sentiment
  (Negative / Neutral / Positive)
- Suggested Solution
  (Generated using RAG system with FAQ knowledge base)

➢ **The output is returned as a structured JSON response via REST API.**

7. Risk Analysis

Risk	Mitigation
Wrong Predictions	Human Review
Bias In AI	Diverse Training Data
Model Drift	Continuous Training
OverLoad	Auto Scaling

◆ System Design

1. Client Layer

- Web App
- Mobile App
- Email GateWay

2. Api GateWay

- **Function**
  - ✓ Authentication (extendable)
  - ✓ Request validation
  - ✓ Routing
  - ✓ Error handling
  - ✓ Health check endpoint

➢ **Central Entry Point**

3. Processing Layer

- **Ticket Service**
  - Text Cleaning Service
  - ML prediction Service
  - NLP Sentiment Service
  - RAG Service
- **AI Inference Service**
  - Calls ML Models
- **Knowledge Service**
  - FAQ Matching

4. AI Model Layer

- Category Classification Model
- Priority Classification Model
- Priority Model
- Sentiment Classification Mode
- Retrieval-Augmented Generation Model

➢ **Using a Micro-Model Architecture instead of a single large model.**

5. Data Layer

- Trained ML Models (.pkl)
- FAISS Vector Index
- FAQ Knowledge Base
- Training Datasets
- Logs Storage

6. Presentation Layer

- Agent Dashboard
- Manager Analytics Panel

7. Data Flow

1. User submits ticket via UI
2. FastAPI receives request
3. Text cleaning & preprocessing
4. Category & Priority prediction
5. Sentiment prediction
6. RAG retrieves relevant FAQs using embeddings
7. Ollama generates contextual response
8. JSON response returned

9. UI displays enriched ticket

8.Where AI Models Are Used

AI Type	Component
Supervised ML (Text Classification)	Category Prediction
SuperVised ML	Priority Prediction
SuperVised ML	Sentiment Analysis
Sentence Embeddings + FAISS	FAQ Retrieval
LLM Via Ollama	Response Generation

9.Role of API

- **FastAPI acts as:**
  - Central integration layer
  - Orchestrator between services
  - Real-time inference interface
  - Production-ready REST service
  - Validation layer (Pydantic models)

10.Tech Stack

- **Backend**
  - Python
  - FastAPI
  - Uvicorn
- **Machine Learning**
  - Scikit-learn
  - XGBoost
  - NLTK
  - TF-IDF Vectorizer
- **RAG System**
  - SentenceTransformers (all-MiniLM-L6-v2)
  - FAISS (Vector Similarity Search)

        - Ollama (LLM inference – llama3.2)
    - **Frontend**
        - Streamlit
    - **Model Persistence**
        - Joblib
        - Pickle

◆ AI Lifecycle

1. Data Collection Strategy

- Historical Tickets
- Resolved Cases
- Customer Feedback
- FAQ Documents
- Agent Corrections

2. AI Model Architecture

Model	Technique
Classification	TF-IDF + Supervised ML
Sentiment	TF-IDF + Linear SVM
Priority	TF-IDF + Linear SVM
Retrival	Sentence Embeddings + FAISS

3. Training Pipeline

- Data Collection
- Data Cleaning
- Data Preprocessing
- Feature Engineering
- Model Training
- Evaluation
- Deployment

- Monitoring

4. Production AI Workflow

- Incoming Ticket
- Real-time Inference
- Confidence Score Generation
- Human Review
- Continuous Learning Update
- ➢ **Confidence-Aware Decision System**

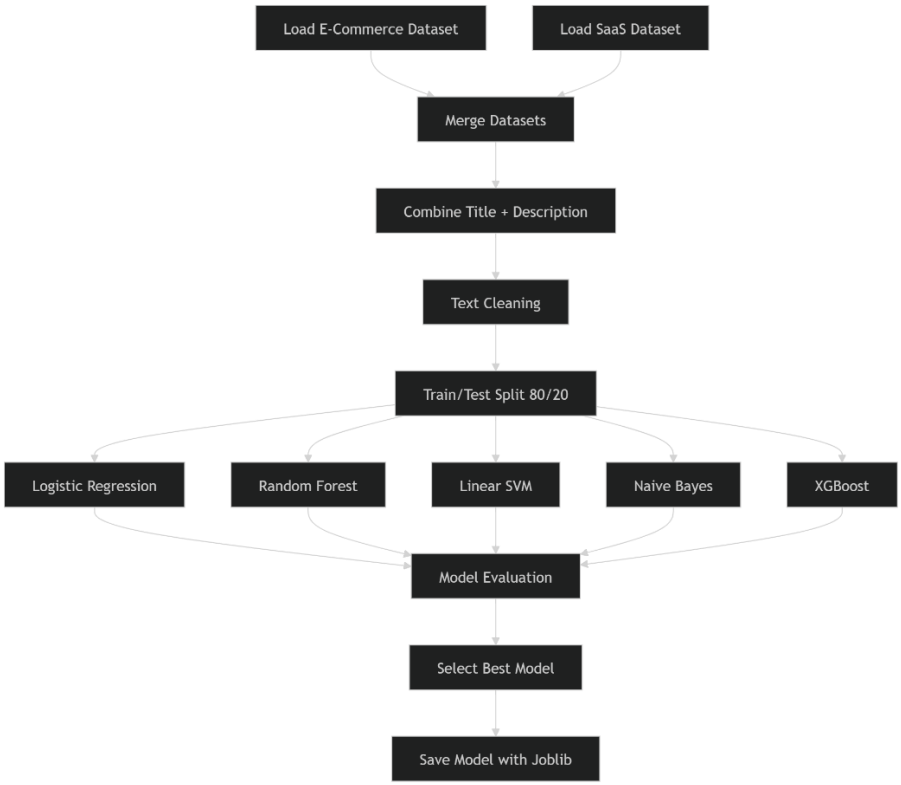
◆ AI System Implementation & Testing

1.Dataset Description

- **Two datasets were used:**
  - E-Commerce Support Dataset
  - SaaS-Tech Support Dataset
- **Total combined dataset size:**
  - ≈ 20,000 tickets
- **Each ticket contains:**
  - Title
  - Description
  - Category
  - Priority
  - Sentiment

➢ **After preprocessing, text was merged and cleaned.**

2.Text Preprocessing



- **Applied steps:**
    - Lowercasing
    - Punctuation removal
    - Number removal
    - Stopwords removal (NLTK)
    - Regex filtering
  - **This ensures cleaner TF-IDF feature extraction.**

3.Category Classification Model

- **Algorithms Tested:**
    - Logistic Regression
    - Random Forest
    - Linear SVM
    - Multinomial Naive Bayes
    - XGBoost
- **Best Performing Models:**
    - Logistic Regression & Linear SVM
- **Final Accuracy:**
    - 97.27%
- **Performance (Linear SVM)**
    - **Accuracy:** 0.97275
    - **Macro F1-score:** 0.97
- **This model was saved as:**
    - category_pipeline_lr.pkl

4.Sentiment Classification Model

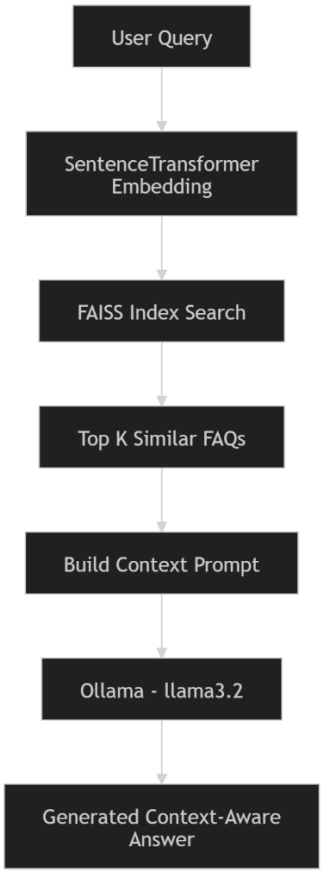
- **3 classes:**
    - Negative
    - Neutral
    - Positive
- **Algorithms Tested:**
    - Logistic Regression

    - Random Forest
    - Linear SVM
    - Multinomial Naive Bayes
    - XGBoost
- **Best Performing Model:**
    - Linear SVM
- **Final Accuracy:**
    - 63.85%
- **Observation:**
    - Class imbalance exists (positive class underrepresented).
    - Despite that, SVM performed better than other models.
- **Saved as:**
    - sentiment_pipeline_lsvm.pkl

5.Priority Classification Model

- **3 classes:**
    - Low
    - Medium
    - High
- **Best Performing Model:**
    - Linear SVM
- **Final Accuracy:**
    - 60.17%
- **Observation:**
    - Priority prediction is more complex because it depends on implicit context rather than explicit keywords.
- **Saved as:**
    - priority_pipeline_lsvm.pkl

6.RAG Implementation



- **Step 1: Embedding Generation**
    - **Model:** all-MiniLM-L6-v2
    - Each FAQ (question + answer) converted into 384-dim embeddings.
- **Step 2: FAISS Index**
    - IndexFlatL2 used for similarity search.
- **Step 3: Retrieval**
    - Top 3 most similar FAQs retrieved using vector similarity.
- **Step 4: Generation**
    - Ollama (llama3.2) generates contextual response based on retrieved FAQs.
    - This improves answer relevance and prevents hallucination.

7.API Implementation

- **Framework:**
    - FastAPI
- **Endpoints:**
    - GET /health
    - POST /ticket
- **Example Request:**
    - {"title":"Payment charged twice","description":"I was charged two times but got only one order."}
- **Example Response:**
    - {"category":"billing","priority":"high","sentiment":"negative","suggested_solution":"Please check refund status..."}

8.System Integration

- **All components integrated as follows:**
    - ML models loaded using Joblib
    - RAG system initialized once (singleton pattern)
    - API orchestrates services
    - UI communicates with backend via HTTP
- **System runs locally using:**
    - uvicorn app:app –reload
    - streamlit run streamlit_app.py

9.Testing Strategy

- **Model Testing**
    - Train/Test Split (80/20)
    - Stratified sampling
    - Evaluation metrics:
        - ✓ Accuracy
        - ✓ Precision
        - ✓ Recall
        - ✓ F1-Score
        - ✓ Confusion Matrix

- **API Testing**
    - Manual testing via Streamlit UI
    - Health endpoint validation
    - Error handling tested (invalid input / backend down)
- **Functional Testing Cases**

Expected Result	Case
Billing category	Refund issue
Technical category	App crash
Negative sentiment	Angry tone
Account category	Password reset

◆ Final System Capabilities

- **The final system:**
    - Automatically classifies tickets
    - Predicts urgency
    - Detects customer sentiment
    - Retrieves knowledge-based answers
    - Generates LLM-enhanced responses
    - Provides real-time inference
    - Supports modular AI lifecycle

◆ Future Enhancements

- Voice Ticket Support
- Multilingual AI
- Auto-Reply System
- Predictive Customer Issues
- Real-Time Chatbot Integration